Guidance for C++ II Midterm and Final Projects

Overview
This document serves as a comprehensive guide for students undertaking the C++ II midterm or final project. The projects are divided into three main deliverables: a planning and preparation document, the C++ code files, and a video walkthrough of the code. Below are detailed instructions for each deliverable.

Planning and Preparation Document

Objective: The planning and preparation document is essential for outlining your approach to the project before diving into coding.

Contents:

- **Summary of the Problem:**
  - Provide a clear and concise description of the problem you are addressing in your project. Explain the context and significance of the problem.
- **Input and Output Description:**
  - Describe what inputs your program will receive and what outputs it will produce. Avoid using variable names; focus on the type of data and any relevant formats. Remember to describe the purpose of the input and output data and not just list names and data types. Discuss any constraints on data values, why they are needed in this program, etc.

Example:

- Input: A list of integers representing user scores on course assessments. Scores are between 0 and 100. (Your document will obviously have more input values)
- Output: A single integer representing the average score, and is used to compute that letter grade (Your document will obviously have more output values)
- **Key Algorithmic Challenges:**
  - Identify algorithmic challenges you anticipate encountering during implementation. Discuss how you plan to address these challenges and include pseudocode where appropriate.

Example Pseudo Code of a function:

```
function calculateAverage(scores):
 total = 0
 for each score in scores:
   total += score
 return total / length(scores)
```

Notice how the pseudo code does not strictly follow the syntax rules of any programming language. It is more about straightforwardly conveying the algorithm's logic. In contrast, C++ has a strict syntax that must be followed, including the use of semicolons, braces, and specific data types.

Pseudo code often uses generic terms like total and scores without specifying data types. In C++, you must declare the data types explicitly (e.g., int, float, std::vector<int>). Control structures in pseudo code are presented in a more readable format (such as for each) without specific syntax. In C++, control structures like loops require specific keywords (for, while) and syntax (including parentheses and braces).

In pseudo code, the function does not require return type declarations. In C++, you must define the return type (e.g., float calculateAverage(std::vector<int> scores)).

Most importantly, pseudo code prioritizes clarity and ease of understanding, while C++ prioritizes precision and correctness in execution. Pseudo code is a bridge between the problem-solving stage and the actual code you write.

C++ Code Files

Objective: The actual coding phase where students implement their project in C++.

Submission Guidelines:

- **Submit all the individual .cpp and .h files separately; do not compress them into a single file.**

Internal Documentation Guidelines:

1. **File Headers:**
   - Each file must contain a comment header that includes:
   - Description of the file's contents (classes, functions, variables).
   - An explanation of the file's purpose within the software.
2. **Class Documentation:**
   - Each class declaration should include comments that describe its purpose and necessity.
3. **Function Documentation:**
   - Each function must have comments explaining:
   - Its purpose and necessity.
   - The purpose of each parameter.
   - The purpose of the return value.
4. **Standard Library Usage:**
   - Whenever you use an object, method, or function from the standard library, explain in a comment why it was chosen and what problem it solves.

Note: Your code may seem "over-commented," but this is a beneficial learning exercise that showcases your ability to articulate your thought process.

Video Walkthrough

Objective: The video should provide an in-depth exploration of your code, highlighting key components and demonstrating functionality. You want to be able to prove to the person watching your video that you are an expert with the code you wrote. Don't just hand-wave over the functionality.

Video Guidelines:

- **Code Walkthrough:**
  - Focus on a few particularly interesting or complex sections of your program. Explain these areas in detail, almost line-by-line.
- **Demonstrate Functionality:**
  - Show executions of your program, covering various functionalities and behaviors, including special cases and error conditions.
  - It may be helpful to explain some functionality in the code, followed by an execution of the program showing off that aspect. This is particularly useful when you have an independent set of functionality that can be easily shown.
- **Preparation:**
  - Plan your video before filming. Avoid improvisation to ensure clarity and coherence.

Note: These videos may seem long, 30 minutes or so, but that is necessary to explain your code and the output.

As you embark on this project, remember that it's an opportunity to learn and grow as a developer. Embrace the challenges, enjoy the creative process, and take pride in your work. Your efforts will pay off, and the skills you gain will be invaluable in your future endeavors. Good luck and have fun coding!